MULTI MANAGEMENT SYSTEM

Python Project Documentation

Project Type	Python GUI and Console Application
Modules	Student Library and Inventory Management
Technology	Python Tkinter and JSON
Submitted By	Rushikesh Laxman Hadawale

A compact system for managing everyday records through one application.

Project Overview

The Multi Management System is a beginner friendly Python application that combines student, library, and inventory record management in one program. It provides a menu driven console interface and a simple Tkinter desktop interface. Both interfaces use the same core logic and save data locally, so users can continue working with the records after the program is closed and reopened.

Purpose and Objectives

- Create one simple program for managing three common types of records.
- Apply core Python concepts such as classes, functions, dictionaries, lists, file handling, exception handling, and regular expressions.
- Provide add, view, search, update, and delete operations for each module.
- Validate user input and prevent duplicate record IDs.
- Store records safely in JSON files without requiring an external database or package.

Technology Used

Technology	Use in the Project
Python 3.10 or later	Application logic, validation, storage, and console interaction
Tkinter and ttk	Tabbed desktop user interface
JSON	Persistent local storage for students, books, and items
pathlib	Reliable handling of data-file paths
re	Email format validation for student records

System Modules

Module	Record Details	Main Functions
Student Management	Student ID, name, age, course, email	Add, view, search, update, delete, course summary
Library Management	Book ID, title, author, quantity	Add, view, search, update, delete
Inventory Management	Item ID, item name, category, quantity, price	Add, view, search, update, delete

Program Design

The project separates the interface from the data-management logic. The main.py file runs the console menu. The gui.py file creates the desktop interface. The student_manager.py file contains the reusable classes that validate records, perform CRUD operations, and read or write JSON files. This separation makes the program easier to understand, test, and extend.

Project File Structure

File or Folder	Responsibility
main.py	Starts the menu driven console application.
gui.py	Starts the Tkinter desktop application with three tabs.
student_manager.py	Defines StudentManager and RecordManager classes.
data/students.json	Stores student records after they are added.
data/books.json	Stores library book records after they are added.
data/items.json	Stores inventory records after they are added.

Class Design

StudentManager manages student records and applies student-specific rules. It checks all required fields, normalizes names and course names, requires ages from 3 to 120, validates email addresses, and converts student IDs to uppercase. RecordManager is a reusable class used by the Library and Inventory modules. It validates required fields, checks quantities and prices, prevents duplicate IDs, and manages the shared CRUD workflow.

Core Workflow

Step	What Happens
1. Enter data	The user enters record details in the console or the desktop form.
2. Validate	The manager checks required fields, IDs, email, age, quantity, and price as applicable.
3. Save	A valid record is added to the in-memory list and written to the relevant JSON file.
4. Retrieve	The user can view, search, select, update, or delete stored records.

Functional Features

Student Management

Student records include a student ID, name, age, course, and email address. The system rejects empty values, duplicate IDs, invalid email formats, and ages outside the accepted range. Searches can match the ID, name, course, or email address. Records are displayed in student ID order.

Library Management

The library module keeps book ID, title, author, and quantity information. It prevents duplicate book IDs and checks that quantity is a valid non-negative number. A search checks every saved value, making it easy to locate a book by its ID, title, or author.

Inventory Management

The inventory module stores an item ID, item name, category, quantity, and price. It requires complete entries and validates numeric quantity and price values. Prices are rounded to two decimal places before saving.

Desktop Interface

The Tkinter interface presents separate Students, Library, and Inventory tabs. Each tab contains a record-details form, action buttons, a live search field, record count, and a table of saved records. Selecting a table row loads its values into the form so the record can be updated or deleted.

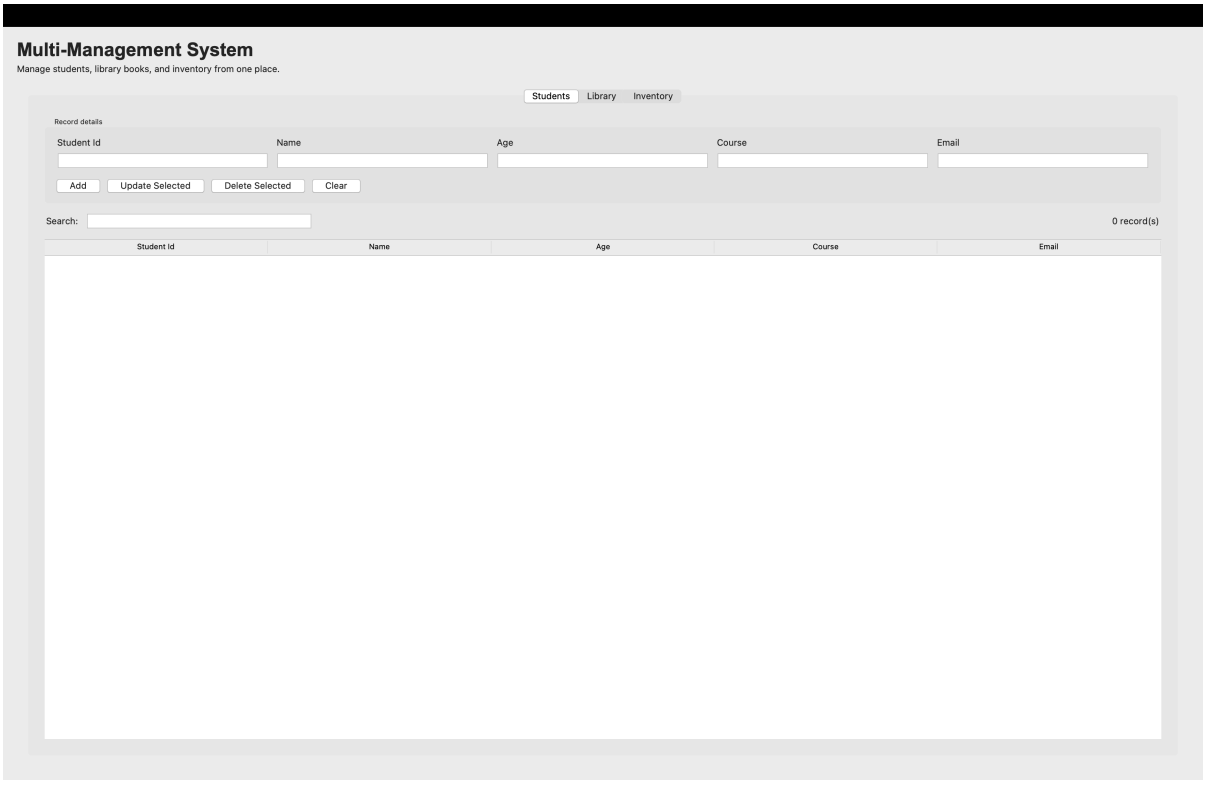


Figure 1 Multi Management System desktop interface

Data Storage and Validation

The application creates the data folder when saving the first record. Each module uses its own JSON file, which keeps the records readable and separate. At startup, the manager loads the existing file. If no file exists, or if the file cannot be read as valid JSON, the program starts with an empty record list rather than stopping unexpectedly.

Validation Rule	Applied To	Result
Required fields	All modules	Incomplete records are rejected.
Unique ID	All modules	Duplicate student, book, or item IDs are rejected.
Age 3 to 120	Students	Invalid or non-numeric ages are rejected.
Email pattern	Students	An invalid email address is rejected.
Non-negative numbers	Library and Inventory	Invalid quantities or prices are rejected.

How to Run the Program

Install Python 3.10 or later. Tkinter is normally included with standard Python installations. Open a terminal in the project folder and choose either of the following commands.

Interface	Command	Expected Result
Console application	python3 main.py	Displays the main menu and module options.
Desktop application	python3 gui.py	Opens the tabbed Multi Management System window.

Testing

The project supports automated testing through Python unittest. If a tests folder is included with the project, the test suite can be run with python3 -m unittest discover -s tests -v. Manual testing should also verify adding, searching, updating, and deleting records in all three modules, plus checks for duplicate IDs and invalid input.

Conclusion

The Multi Management System demonstrates how Python can be used to build a practical record-management application. It combines object-oriented design, reusable CRUD logic, input validation, file persistence, a console menu, and a graphical user interface. The code structure allows the application to be extended later with features such as login roles, report generation, sorting choices, export options, or database storage.